Multi-Client Functional Encryption for Small Domains

Suvasree Biswas, Mohit Vaid, and Arkady Yerukhimovich

George Washington University
suvasree@gwu.edu m.vaid@gwu.edu arkady@gwu.edu

Abstract. In this paper, we revisit the problem of multi-client functional encryption (MCFE) for general functions. Specifically, we consider the setting of private-key MCFE for constant-arity functions where the input domain is polynomial in the security parameter. Surprisingly, we show that in this setting it is possible to construct a private-key MCFE scheme secure for a bounded number of key and encryption queries based only on the minimal assumption that one-way functions exist. In contrast, all prior constructions of MCFE for general functions require very strong assumptions such as indistinguishability obfuscation or multilinear maps.

Our main technique is to show that private-key MCFE for polynomial input domain can be built from any private-key multi-input functional encryption (MIFE) while inheriting the security properties of the underlying MIFE. Instantiating our construction with the MIFE of Brakerski et al. (Eurocrypt 2016) gives us a construction based only on the existence of one-way functions.

Keywords: Functional Encryption · Multi-Client Functional Encryption · Multi-Input Functional Encryption

1 Introduction

Originating from a sequence of foundational works [SW05, GPS+06, BW07, KSW08, LOS+10, ONe10, BSW11], functional encryption goes beyond the conventional "all-or-nothing" security guarantee of classical encryption. Specifically, functional encryption allows the issuance of a functional key corresponding to a function f that allows evaluation of the function f on encrypted data, while hiding everything else. Over the past decade, substantial effort has been devoted to studying both the security guarantees and the efficiency of functional encryption schemes [BSW11, ONe10, GVW12, AGV+13, BO13, BCP14, DIJ+13, GGH+16, GKP+13].

While traditional, single-input, functional encryption has many applications it has a critical limitation—the entire input must be encrypted at one time into a single ciphertext. However, it is often desirable to evaluate functions over multiple ciphertexts for example if multiple pieces of data are encrypted over a long period of time, or if the encryptions are done by multiple different parties.

To address this limitation, Goldwasser et al. [GGG+14] formalized two natural extensions: multi-input functional encryption (MIFE) and multi-client functional encryption (MCFE). MIFE allows the evaluation of multi-input functions over a set of ciphertexts, but all of these must be encrypted with the same encryption key, which must be kept secure. On the other hand, MCFE allows the input ciphertexts to be generated using different encryption keys, even allowing some of these keys to be controlled by an adversary. In both of these, the goal is to guarantee that nothing beyond the value of f on the encrypted inputs is revealed.

The early constructions of MCFE and MIFE for general functionalities, initiated by Goldwasser et al.[GGG+14], relied on the heavy cryptographic machinery of indistinguishability obfuscation [BGI+01, GGH+16, KMN+14] or multilinear maps [BLR+15]. Moreover, [BV18, AJS15, KNT18, GGG+14] showed that these assumptions are, in fact, *necessary* as unbounded variants of functional encryption imply obfuscation. Thus, a fundamental question arises whether there is any version of MIFE and MCFE for general functions that can be realized from more standard, lighter assumptions.

Surprisingly, Brakerski et al. [BKS18] showed that for the case of private-key MIFE (i.e. a variant of MIFE where the master secret key is needed to encrypt) one can construct such a scheme for constant-arity functions that is secure against an adversary making a bounded number of key and encryption queries from the minimal assumption of one-way functions. However, this result did not give a construction of MCFE and to date, the only constructions of secure MCFE are for restricted function classes such as inner products [ABG19, LT19, SV23, NPP22, NPP23, NPS24, NPP25] and separable functions [CSW21].

In this paper, we revisit the question of constructing MCFE from weaker assumptions and ask:

> Is there a variant of MCFE for general functions that can be instantiated using only standard assumptions?

1.1 Our Contribution

We answer the above question in the affirmative. Our work departs from the prevailing paradigm for constructing MCFE schemes: Instead of limiting the class of admissible functions, we demonstrate that one can retain general functionality by restricting the input domain.

Specifically, we focus on the case of private-key MCFE for all constant-arity functions computable by polynomial size circuits. In this setting, we show how to construct an MCFE scheme with polynomial size input domain that is secure against an adversary making a bounded number of key and encryption queries from *any* MIFE scheme. In particular, our construction only needs the MIFE scheme to be secure against an adversary making the same number of key-generation queries and polynomially more encryption queries than the resulting MCFE scheme. Specifically, we prove the following theorems:

Theorem 1 (Informal). *A private-key* MIFE *for general constant-arity functions that is secure against an adversary making a bounded number of encryption*

and key-generation queries implies a private-key MCFE *for general constant-arity functions over a polynomial input domain that is secure against a similarly bounded adversary.*

Instantiating the MIFE in Theorem 1 with the private-key MIFE construction by Brakerski et al. [BKS18], we get the following corollary.

Corollary 1 (Informal). *Assuming the existence of one-way functions, there exists a private-key* MCFE *scheme for general constant-arity functions over polynomial size domain that is secure against an adversary making a bounded number of key-generation and encryption queries.*

1.2 Our Approach

Observe that both MCFE and MIFE have essentially the same evaluation functionality. Given a functional key, and a set of ciphertexts, this procedure evaluates the function on the encrypted messages. Thus, an immediate idea is to somehow allow the MCFE clients to produce MIFE ciphertexts for their messages so that the MIFE evaluation procedure can be used. The main challenge in achieving this is that in MIFE all encryptions are generated under the same (secret) key, while in MCFE every client must receive its own encryption key, and moreover the scheme must remain secure even when the adversary knows some of the encryption keys. It is thus not clear how one can use the MCFE encryption keys to generate ciphertexts under one MIFE key.

Our construction bypasses this challenge by leveraging the fact that the input domain is of polynomial size. Specifically, for a polynomial size domain, the trusted key authority can pre-encrypt MIFE ciphertexts for every possible message at every input slot. Since we are considering constant-arity functions, there are only polynomially many ciphertexts to generate. We then assign each client in MCFE to one of these input slots and give the corresponding ciphertexts to each client as their encryption key. With this key, a client can encrypt a message m by just choosing the mth ciphertext from its key. This allows the clients to generate MIFE ciphertexts for their messages without needing any information about the MIFE secret key. Evaluation can then just use the MIFE key generation and evaluation procedure.

However, we are not quite done. The problem with the above construction is that the MCFE encryption procedure is deterministic, so each client can only encrypt each message exactly once. To deal with this limitation, we turn to the notion of labeled MCFE. In labeled MCFE each ciphertext additionally has a label and evaluation can only be done on ciphertexts with the same label thus preventing mix-and-match attacks where an adversary evaluates the function on all possible combinations of encrypted messages. To enable labels in our construction, we simply pre-generate cipheretexts for all message, label pairs–this is still polynomial if the number of possible labels is also small. Now, each client can encrypt a message for each label allowing a bounded number of encryptions. To ensure that labels are used correctly in evaluation, we modify the function in

key generation queries to first check that all labels are equal before producing any output. The resulting MCFE scheme supports a bounded number of encryption and key-generation queries and inherits its security guarantees from the underlying MIFE construction.

Limitations Our construction does have one critical limitation. Like other labeled MCFE schemes, security only holds if clients never reuse a label for more than one encryption. In traditional MCFE [GKL+13, CDG+18, ABK+19] this is handled by using a timestamp or a hash output as the label, thus guaranteeing uniqueness. However, since our label space is small and must be fixed at initialization, we cannot use a randomly chosen label. Instead, we need the clients to be stateful to keep track of the labels they have already used, e.g., via a counter, or we need the application to pre-specify how clients can agree on a label from the pre-generated set in a deterministic way. This also implies that an encryption key can only be used a bounded number of times before it needs to be refreshed. We leave the question of how to remove this statefullness requirement as an interesting open question.

2 Applications

In this section we briefly discuss some potential application of our MCFE construction. We specifically focus on applications where small input domain and the bounded encryptions are a natural fit.

Electronic Voting : One natural application of our construction is for electronic voting where the final winner can be determined via MCFE evaluation of the client's votes. In this setting, the input domain i.e., the set of candidates - is inherently small and fixed (For future candidates, we can just use a number which will match the order in which candidates appear in the poll). Similarly, future election dates are predetermined and can naturally serve as labels in our framework, and it is easy for the clients to know the correct label at voting time. Moreover, the number of eligible voters is fixed in advance, which aligns directly with the arity of the underlying function.

Surveys: Surveys routinely ask respondents to choose from a small set of options or provide a ranking (e.g., from 1 to 10) for some item or activity. Our MCFE allows small-scale surveys to be carried out while maintaining the privacy of individual responses. In particular, our MCFE would enable evaluation of any statistics on the survey responses. Here the label for each survey can be the number of the survey and surveys naturally have a small domain.

Multi-party Circuit PSI: Finally, our MCFE can be used to realize multi-party Circuit-PSI, e.g. [HEK12] where the items in the sets come from a small domain, e.g., birthdays. Using our MCFE for this application gives a solution based only on private-key assumption and with no interaction needed after the initial ciphertexts are sent. Furthermore, because the underlying MIFE scheme is

proven secure against malicious evaluators, this security extends to the circuit PSI construction, offering strong protection against malicious behavior during PSI evaluation.

3 Preliminaries

We summarize the notation and definitions used in this work.

Notation We let $\kappa \in \mathbb{N}$, denote the security parameter. A function $\nu : \mathbb{N} \to \mathbb{N}$ is negligible if $\nu(\kappa) < \frac{1}{\kappa^c}$ for all $c > 0$ and sufficiently large κ. We use $[n] := \{1, \ldots, n\}$. We let t be the total number of clients, whereas $\mathcal{CP}$ and $\mathcal{H}$ denote the sets of corrupted and honest clients respectively. For a fixed constant $t > 1$, we let $\mathcal{F} = \{\mathcal{F}_\kappa\}_{\kappa \in \mathbb{N}}$ represent the set of all functions computable by polynomial size circuits where every $f \in \mathcal{F}_\kappa$ is of the form $f : \mathcal{X}_\kappa^t \to \mathcal{Y}_\kappa$ where $\mathcal{X}_\kappa^t$, $\mathcal{Y}_\kappa$ are the input space and output space respectively and $|\mathcal{X}_\kappa| < \text{poly}(\kappa)$.

3.1 Private Key Multi-Input Functional Encryption [BS18, AJS15]

For any $\kappa, t \in \mathbb{N}$, let $\mathcal{X} = \{\mathcal{X}_\kappa\}_{\kappa \in \mathbb{N}}$ be the plaintext space, $\mathcal{Y} = \{\mathcal{Y}_\kappa\}_{\kappa \in \mathbb{N}}$ be the output space and $\mathcal{F} = \{\mathcal{F}_\kappa\}_{\kappa \in \mathbb{N}}$ be the collection of all functions computable by polynomial size circuits where every f is of the form $f : \mathcal{X}_\kappa^t \to \mathcal{Y}_\kappa$. A private key t-input MIFE for $\mathcal{F}$ consists of quadruple of probabilistic polytime algorithms described as follows.

- $\mathsf{Setup}(1^\kappa)$: On input the unary encoding of the security parameter κ, it outputs a master secret key msk.
- $\mathsf{Enc}(\mathsf{msk}, x_l, l)$: On input the master secret key msk, a message $x_l \in \mathcal{X}_\kappa$ for some slot $l \in t$, the encryption algorithm outputs a ciphertext ct_{x_l}.
- $\mathsf{KG}(\mathsf{msk}, f)$: On input a master secret key msk and a function $f \in \mathcal{F}_\kappa$, it outputs a functional key sk_f.
- $\mathsf{Eval}(\mathsf{sk}_f, (\mathsf{ct}_{x_l})_{l \in [t]})$: On input a functional key sk_f and t ciphertexts $(\mathsf{ct}_{x_l})_{l \in [t]}$ outputs a string $y \in \mathcal{Y}_\kappa \cup \{\perp\}$.

Definition 1 (Correctness). *A private key t-input multi-input functional encryption scheme $\mathsf{MIFE} = (\mathsf{Setup}, \mathsf{KG}, \mathsf{Enc}, \mathsf{Eval})$ for $\mathcal{F}_\kappa$ is correct if there exists a negligible function $\nu(\cdot)$ such that for every $\kappa \in \mathbb{N}$, for every $f \in \mathcal{F}_\kappa$, and for every $(x_l)_{l \in [t]} \in \mathcal{X}_\kappa^t$, it holds that*

$$\Pr[\mathsf{Eval}(\mathsf{sk}_f, (\mathsf{Enc}(\mathsf{msk}, x_l, l))_{l \in [t]}) = f((x_l)_{l \in [t]})] \geq 1 - \nu(\kappa)$$

where $\mathsf{msk} \leftarrow\!\!\$\, \mathsf{Setup}(1^\kappa)$, $\mathsf{sk}_f \leftarrow\!\!\$\, \mathsf{KG}(\mathsf{msk}, f)$, and the probability is taken over the internal random coins of $\mathsf{Setup}, \mathsf{Enc}$ and KG.

In Figure 1 (Left), we present the adaptive security game for MIFE [BS18, GGG+14]. A stateful ppt adversary $\mathcal{A}$ interacts with a challenger that, upon receiving 1^κ, generates a master secret key and samples a hidden bit b. $\mathcal{A}$ is then

$\underline{\mathbf{G}^{\mathrm{adp-ind}}_{\mathsf{MIFE},\mathcal{A}}(1^\kappa)}$

INITIALIZE(1^κ):
1 $b \leftarrow\!\$\ \{0,1\}$
2 $\mathsf{msk} \leftarrow\!\$\ \mathsf{Setup}(1^\kappa)$

ENCO$\left(\left(x_l^0, x_l^1\right)_{l\in[t]}\right)$:
3 $\forall l \in [t]: \quad \mathsf{ct}^b_{x_l} \leftarrow\!\$\ \mathsf{Enc}_{\mathsf{msk}}(x_l^b, l)$
4 Return $(\mathsf{ct}^b_{x_l})_{l\in[t]}$

KEYGENO(f^0, f^1):
5 $\mathsf{sk}_{f^b} \leftarrow\!\$\ \mathsf{KG}_{\mathsf{msk}}(f^b)$
6 Return sk_{f^b}

FINALIZE(b'):
7 Return $b' = b$

$\underline{\mathbf{G}^{\mathrm{adp-ind}}_{\mathsf{MCFE},\mathcal{A}}(1^\kappa)}$

INITIALIZE$(1^\kappa, \mathcal{CP})$:
1 $b \leftarrow\!\$\ \{0,1\}$
2 $(\mathsf{msk}, (\mathsf{ek}_l)_{l\in[t]}) \leftarrow\!\$\ \mathsf{Setup}(1^\kappa)$
3 Return $(\mathsf{ek}_l)_{l\in\mathcal{CP}}$

ENCO $\left((x_l^0, \mathsf{lb}_l), (x_l^1, \mathsf{lb}_l)\right)_{l\in\mathcal{H}}$):
4 $\forall l \in \mathcal{H}:$
$\quad \mathsf{ct}^b_{x_l,\mathsf{lb}_l} \leftarrow\!\$\ \mathsf{Enc}_{\mathsf{ek}_l}(x_l^b, l, \mathsf{lb}_l)$
5 Return $(\mathsf{ct}^b_{x_l,\mathsf{lb}_l})_{l\in\mathcal{H}}$

KEYGENO(f^0, f^1):
6 $\mathsf{sk}_{f^b} \leftarrow\!\$\ \mathsf{KG}_{\mathsf{msk}}(f^b)$
7 Return sk_{f^b}

FINALIZE(b'):
8 Return $b' = b$

Fig. 1: Games defining Adaptive Security of MIFE (Left) and MCFE(Right)

allowed to to make encryption queries (to ENCO) and key-generation queries (to KEYGENO) receiving ciphertexts and function keys respectively. Finally, $\mathcal{A}$ outputs a guess bit b' and wins if $b' = b$. In what follows, we only consider a bounded adversary who makes an apriori-bounded number of oracle queries. We now formalize a valid bounded-MIFE adversary.

Definition 2 (Valid (q_1, q_2)-bounded adversary). *A probabilistic polynomial-time algorithm $\mathcal{A}$ is a* valid (q_1, q_2)-bounded adversary *if there exist bounds $q_1, q_2 \in \mathbb{N}$ such that the following holds. For every private-key t-input multi-input functional encryption scheme $\mathsf{MIFE} = (\mathsf{Setup}, \mathsf{KG}, \mathsf{Enc}, \mathsf{Eval})$ over input space $\mathcal{X} = \{\mathcal{X}_\kappa^t\}$ and function space $\mathcal{F} = \{\mathcal{F}_\kappa\}$, the adversary makes at most q_1 key-generation queries $(f_i^0, f_i^1) \in \mathcal{F}_\kappa$ where $i \in [q_1]$ and at most q_2 encryption oracle queries $\{(x_{j,l}^0, x_{j,l}^1)_{l\in[t]}\} \in (\mathcal{X}_\kappa \times \mathcal{X}_\kappa)^t$ where $j \in [q_2]$. Additionally, to avoid trivial attacks, we require that for all pairs of queries $i \in [q_1]$ and $j \in [q_2]$, it must hold that*

$$f_i^0\left((x_{j,l}^0)_{l\in[t]}\right) \;=\; f_i^1\left((x_{j,l}^1)_{l\in[t]}\right).$$

We can now provide a formal security definition for adaptively-secure MIFE against a valid (q_1, q_2)-bounded adversary.

Definition 3 (adaptive security). *For all sufficiently large $\kappa \in \mathbb{N}$, a private-key t-input multi-input functional encryption scheme $\mathsf{MIFE} = (\mathsf{Setup}, \mathsf{KG}, \mathsf{Enc}, \mathsf{Eval})$ over a message space $\mathcal{X} = \{\mathcal{X}_\kappa\}_{\kappa\in\mathbb{N}}$ and a function space $\mathcal{F} = \{\mathcal{F}_\kappa\}_{\kappa\in\mathbb{N}}$ is* (q_1, q_2)-adaptively secure *if for every* valid (q_1, q_2)-bounded adversary $\mathcal{A}$ *there exists a negligible function $\nu(\cdot)$ such that for $\mathbf{G}^{\mathrm{adp-ind}}_{\mathsf{MIFE},\mathcal{A}}$ as defined in Figure 1(Left) the following holds*

$$\mathbf{Adv}^{\mathrm{adp-ind}}_{\mathsf{MIFE},\mathcal{A}}(1^\kappa) = \left|\Pr[\mathbf{G}^{\mathrm{adp-ind}}_{\mathsf{MIFE},\mathcal{A}}(1^\kappa) \Rightarrow 1] - \frac{1}{2}\right| \leq \nu(\kappa)$$

3.2 Private Key Multi-Client Functional Encryption [GGG+14, GKL+15]

For any $\kappa, t \in \mathbb{N}$, let plaintext space $\mathcal{X} = \{\mathcal{X}_\kappa\}_{\kappa\in\mathbb{N}}$, label space $\mathcal{L} = \{\mathcal{L}_\kappa\}_{\kappa\in\mathbb{N}}$, output space $\mathcal{Y} = \{\mathcal{Y}_\kappa\}_{\kappa\in\mathbb{N}}$ and $\mathcal{F} = \{\mathcal{F}_\kappa\}_{\kappa\in\mathbb{N}}$ be the collection of all functions computable by polynomial size circuits where every $f \in \mathcal{F}_\kappa$ is of the form $f : \mathcal{X}_\kappa^t \to \mathcal{Y}_\kappa$. A private-key t-input multi-client functional encryption scheme MCFE for $\mathcal{F}$ consists of four ppt algorithms described as follows.

- $\mathsf{Setup}(1^\kappa)$: On input the unary encoding of the security parameter κ, it outputs master secret key msk and t encryption keys $(\mathsf{ek}_l)_{l\in[t]}$.
- $\mathsf{Enc}(\mathsf{ek}_l, x_l, l, \mathsf{lb})$: On input an encryption key ek_l for some slot $l \in [t]$, a message $x_l \in \mathcal{X}_\kappa$, a label $\mathsf{lb} \in \mathcal{L}_\kappa$, it outputs a ciphertext $\mathsf{ct}_{x_l,\mathsf{lb}}$.
- $\mathsf{KG}(\mathsf{msk}, f)$: On input a master secret key msk and a function $f \in \mathcal{F}_\kappa$, it outputs a functional key sk_f.
- $\mathsf{Eval}(\mathsf{sk}_f, (\mathsf{ct}_{x_l,\mathsf{lb}})_{l\in[t]})$: On input a functional key sk_f and t ciphertexts $(\mathsf{ct}_{x_l,\mathsf{lb}})_{l\in[t]}$ it outputs a string $y \in \mathcal{Y}_\kappa \cup \{\perp\}$.

Definition 4 (Correctness). *A private key t-input multi-client functional encryption scheme* MCFE *for $\mathcal{F}$ is correct if there exists a negligible function $\nu(\cdot)$ such that for every $\kappa \in \mathbb{N}$, for every $f \in \mathcal{F}_\kappa$, and for every $(x_l)_{l\in[t]} \in \mathcal{X}_\kappa^t$, $\mathsf{lb} \in \mathcal{L}_\kappa$, it holds that*

$$\Pr[\mathsf{Eval}(\mathsf{sk}_f, (\mathsf{Enc}(\mathsf{ek}_l, (x_l, l, \mathsf{lb}))_{l\in[t]}) = f((x_l)_{l\in[t]})] \geq 1 - \nu(\kappa)$$

where $\mathsf{msk}, (\mathsf{ek}_l)_{l\in[t]} \leftarrow\!\!\$\ \mathsf{Setup}(1^\kappa)$, $\mathsf{sk}_f \leftarrow\!\!\$\ \mathsf{KG}(\mathsf{msk}, f)$, *and the probability is taken over the internal random coins of* $\mathsf{Setup}, \mathsf{Enc}$ *and* KG.

In Figure 1(Right), we define adaptive security for MCFE [GGG+14, GKL+15, SV23]. A stateful ppt adversary $\mathcal{A}$ declares a corrupted set $\mathcal{CP}$, receives encryption keys for all parties in set $\mathcal{CP}$, and may query EncO queries to get encryptions of messages from honest parties, and KeygenO to obtain function keys. Finally, $\mathcal{A}$ outputs a guess b' and wins if $b' = b$. As we did for MIFE, we focus on the case of an adversary making a bounded number of oracle queries. We formalize such a valid bounded adversary below.

Definition 5 (Valid (q_1, q_2)-bounded adversary). *A probabilistic polynomial-time algorithm $\mathcal{A}$ is a* valid (q_1, q_2)-bounded adversary *if there exist bounds $q_1, q_2 \in \mathbb{N}$ such that the following holds. For every private-key t-input multi-client functional encryption scheme* $\mathsf{MCFE} = (\mathsf{Setup}, \mathsf{KG}, \mathsf{Enc}, \mathsf{Eval})$ *over input space $\mathcal{X} = \{\mathcal{X}_\kappa^t\}$, label space $\mathcal{L} = \{\mathcal{L}_\kappa\}$ and function space $\mathcal{F} = \{\mathcal{F}_\kappa\}$, the adversary makes at most q_1 key-generation queries $(f_i^0, f_i^1) \in \mathcal{F}_\kappa$, $i \in [q_1]$, and at most q_2 encryption queries: $\{(x_{j,l}^0, x_{j,l}^1, \mathsf{lb}_{j,l})_{l\in\mathcal{H}}\} \in (\mathcal{X}_\kappa \times \mathcal{X}_\kappa \times \mathcal{L}_\kappa)^t$ for $j \in [q_2]$ where $\mathcal{H}$ is the set of honest parties. To ensure consistency of labels, we require that $\mathsf{lb}_{j,l}$ is the same in all l tuples of an encryption query. To avoid trivial attacks, we require that for every $i \in [q_1]$ and $j \in [q_2]$, it holds that*

$$f_i^0\big((x_{j,l}^0)_{l\in\mathcal{H}}, (y_{j,l})_{l\in\mathcal{CP}}\big) = f_i^1\big((x_{j,l}^1)_{l\in\mathcal{H}}, (y_{j,l})_{l\in\mathcal{CP}}\big)$$

for all possible adversarial inputs $(y_{j,l})_{l\in\mathcal{CP}}$.

We now present the formal security definition for adaptively-secure MCFE against a valid (q_1, q_2)-bounded adversary.

Definition 6 (adaptive security). *For all sufficiently large $\kappa \in \mathbb{N}$, a private-key t-input multi-client functional encryption scheme* $\mathsf{MCFE} = (\mathsf{Setup}, \mathsf{KG}, \mathsf{Enc}, \mathsf{Eval})$ *over input space $\mathcal{X} = \{\mathcal{X}_\kappa\}_{\kappa\in\mathbb{N}}$, label space $\mathcal{L} = \{\mathcal{L}_\kappa\}$ and a function space $\mathcal{F} = \{\mathcal{F}_\kappa\}_{\kappa\in\mathbb{N}}$ is* (q_1, q_2)-adaptively secure *if for every* valid (q_1, q_2)-bounded adversary $\mathcal{A}$ *there exists a negligible function $\nu(\cdot)$ such that for $\mathbf{G}^{\mathrm{adp-ind}}_{\mathsf{MCFE},\mathcal{A}}$ as in Figure 1(Right) the following holds*

$$\mathbf{Adv}^{\mathrm{adp-ind}}_{\mathsf{MCFE},\mathcal{A}}(1^\kappa) = \left|\Pr[\mathbf{G}^{\mathrm{adp-ind}}_{\mathsf{MCFE},\mathcal{A}}(1^\kappa) \Rightarrow 1] - \frac{1}{2}\right| \leq \nu(\kappa)$$

4 Construction

In this section, we present a construction of an adaptively secure private-key multi-client functional encryption scheme (MCFE) for constant-arity functions over polynomial-size domains with bounded key-generation and encryption oracle queries, obtained by using, as a black-box, a private-key multi-input functional encryption scheme (MIFE) for constant-arity functions that also supports bounded key-generation and encryption oracle queries. For a security parameter κ and a constant $t > 1$ representing number of parties, consider plaintext space $\mathcal{X} = \{\mathcal{X}_\kappa\}_{\kappa\in\mathbb{N}}$ where $|\mathcal{X}_\kappa| < \mathrm{poly}(\kappa)$, label space $\mathcal{L} = \{\mathcal{L}_\kappa\}_{\kappa\in\mathbb{N}}$ such that $|\mathcal{L}_\kappa| < \mathrm{poly}(\kappa)$ and output space $\mathcal{Y} = \{\mathcal{Y}_\kappa\}_{\kappa\in\mathbb{N}}$. Let $\mathcal{F} = \{\mathcal{F}_\kappa\}_{\kappa\in\mathbb{N}}$ be a set of all functions computable by polynomial size circuits where every $f \in \mathcal{F}_\kappa$ is of the form $f : \mathcal{X}^t_\kappa \to \mathcal{Y}_\kappa$. We provide below the construction of a private-key MCFE scheme for $\mathcal{F}$.

4.1 Our Construction

Our approach, as outlined in Section 1.1, supports polynomial-size message and label spaces. Using the MIFE scheme, we start by generating the MIFE master secret key msk and use it to pre-compute ciphertexts for all (plaintext, label) pairs using the MIFE encryption algorithm, we follow this approach for each slot $l \in [t]$. The generated table of ciphertexts for a slot $l \in [t]$ is then given to each of the t parties as the encryption key $\mathsf{MCFE.ek}_l$ for the MCFE scheme. To encrypt, each party $l \in [t]$, comes up with a message x from the message space $\mathcal{X}_\kappa$ and a label lb from label space $\mathcal{L}_\kappa$ and uses the $\mathsf{MCFE.ek}_l$ to find the ciphertext corresponding to the tuple (x, lb) to get the corresponding MIFE ciphertext.

For any t-ary function f computable by $\mathrm{poly}(\kappa)$ size circuit, the MCFE function key is simply the corresponding MIFE function key for f from $\mathcal{F}_\kappa$. Thereafter, an evaluator holding the functional key $\mathsf{MCFE.sk}_f$ can then compute the functional evaluation on the underlying plaintexts by calling the MCFE.Eval which in turn runs MIFE.Eval, provided that the ciphertexts supplied by all t parties were encrypted under a common label. This restriction is necessary to circumvent an inherent limitation of MIFE, namely the ability of an evaluator to arbitrarily *mix-and-match* ciphertexts of different time stamps during evaluation.

$\mathcal{D}_f\left((x_l, l, \mathsf{lb}_l)_{l\in[t]}\right)$:

1 if $\exists l, l' \in [t]$ *st* $\mathsf{lb}_l \neq \mathsf{lb}_{l'}$: Return $\perp$
2 Return $f(x_1, \ldots, x_t)$

MCFE.Setup(1^κ):

1 $\mathsf{MIFE.msk} \leftarrow\!\!\$\ \mathsf{MIFE.Setup}(1^\kappa)$
2 $\forall l \in [t]$: $sCT_l \leftarrow \emptyset$
3 $\forall m \in \mathcal{X}_\kappa$:
4 $\forall h \in \mathcal{L}_\kappa$:
5 $\mathsf{MIFE.ct}_{l,m,h} \leftarrow\!\!\$\ \mathsf{MIFE.Enc}_{\mathsf{MIFE.msk}}(m, l, h)$
6 $sCT_l = sCT_l \cup \{(m, h, \mathsf{MIFE.ct}_{l,m,h})\}$
7 $\mathsf{MCFE.ek}_l \leftarrow sCT_l$
8 $\mathsf{MCFE.msk} \leftarrow \mathsf{MIFE.msk}$
9 Return $\mathsf{MCFE.msk}, (\mathsf{MCFE.ek}_l)_{l\in[t]}$

MCFE.Enc$(\mathsf{MCFE.ek}_l, x, l, \mathsf{lb})$:

10 parse $\{(m, h, \mathsf{MIFE.ct}_{l,m,h}) : \forall m \in \mathcal{X}_\kappa, \forall h \in \mathcal{L}_\kappa\} \leftarrow \mathsf{MCFE.ek}_l$
11 $\mathsf{MCFE.ct}_{x,\mathsf{lb}} \leftarrow \mathsf{MIFE.ct}_{l,m,h}$ where $m = x$, $h = \mathsf{lb}$
12 Return $\mathsf{MCFE.ct}_{x,\mathsf{lb}}$

MCFE.KG$(\mathsf{MCFE.msk}, f)$:

13 $\mathsf{MIFE.sk}_{\mathcal{D}} \leftarrow\!\!\$\ \mathsf{MIFE.KG}_{\mathsf{MIFE.msk}}(\mathcal{D}_f)$
14 Return $\mathsf{MCFE.sk}_f := \mathsf{MIFE.sk}_{\mathcal{D}}$

MCFE.Eval $\left(\mathsf{MCFE.sk}_f, (\mathsf{MCFE.ct}_{x_l,\mathsf{lb}_l})_{l\in[t]}\right)$:

15 parse $\mathsf{MIFE.sk}_{\mathcal{D}} \leftarrow \mathsf{MCFE.sk}_f$
16 parse $\mathsf{MIFE.ct}_{l,x_l,\mathsf{lb}_l} \leftarrow \mathsf{MCFE.ct}_{x_l,\mathsf{lb}_l}$
17 Return $\mathsf{MIFE.Eval}_{\mathsf{MIFE.sk}_{\mathcal{D}}}\left((\mathsf{MIFE.ct}_{l,x_l,\mathsf{lb}_l})_{l\in[t]}\right)$

Fig. 2: MCFE Construction

As should be obvious from the above description, security of our scheme requires that each client only use each label in at most one ciphertext. This requires clients to ensure that they avoid label reuse. Since our label domain is small, we cannot achieve this by simply sampling labels at random. Thus, either the clients need to maintain a counter to indicate labels they have already used, or there needs to be some external mechanism that can assign labels from the pre-determined label space. Moreover, since our pre-determined label space is small, this also means that an encryption key can only be used a bounded number of times before it must be refreshed.

Construction Details Let $\mathsf{MIFE} = (\mathsf{MIFE.Setup}, \mathsf{MIFE.KG}, \mathsf{MIFE.Enc}, \mathsf{MIFE.Eval})$ be a private-key multi-input functional encryption scheme for $\mathcal{F}$. We define MCFE as a quadruple of ppt algorithms (MCFE.Setup, MCFE.KG, MCFE.Enc, MCFE.Eval), each of which is briefly explained below. The formal construction is given in Figure 2.

$\mathsf{MCFE.Setup}(1^\kappa)$. On input the unary encoding of the security parameter κ, the Setup algorithm first invokes the setup procedure of MIFE to create the master secret key $\mathsf{MIFE.msk}$. For each client slot $l \in [t]$, an initially empty ciphertext table is created. Then, for every plaintext $m \in \mathcal{X}_\kappa$ and label $h \in \mathcal{L}_\kappa$, the algorithm samples fresh randomness and applies the MIFE encryption procedure to generate the corresponding ciphertext. Each resulting ciphertext is stored in the table at index (m, h). The completed table for slot l serves as the client's encryption key $\mathsf{MCFE.ek}_l$. Finally, the algorithm outputs the MIFE master secret key together with the collection of encryption keys $(\mathsf{MCFE.ek}_l)_{l\in[t]}$ of all clients.

$\mathsf{MCFE.Enc}(\mathsf{MCFE.ek}_l, x, l, \mathsf{lb})$. A client with an encryption key $\mathsf{MCFE.ek}_l$, provides for slot l, an input $x \in \mathcal{X}_\kappa$ and a label $\mathsf{lb} \in \mathcal{L}_\kappa$ to the encryption algorithm which then retrieves the precomputed ciphertext for (x, lb) from the table $\mathsf{MCFE.ek}_l$. This ciphertext is returned as the MCFE encryption of x for slot l under label lb.

$\mathsf{MCFE.KG}(\mathsf{MCFE.msk}, f)$. On input a function $f \in \mathcal{F}_\kappa$, we modify f to the related function $\mathcal{D}_f$ that first checks that the labels on all inputs are equal before computing f. Next, key generation uses $\mathsf{MCFE.msk} = \mathsf{MIFE.msk}$, to run the MIFE key generation procedure, i.e., it outputs $\mathsf{MCFE.sk}_f = \mathsf{MIFE.sk}_{\mathcal{D}_f} \leftarrow \mathsf{MIFE.KG}(\mathsf{MIFE.msk}, \mathcal{D}_f)$.

$\mathsf{MCFE.Eval}(\mathsf{MCFE.sk}_f, (\mathsf{MCFE.ct}_{x_l,\mathsf{lb}_l})_{l\in[t]})$. The evaluation algorithm takes as input the functional key $\mathsf{MCFE.sk}_f$ together with a ciphertext from each client. Recall that these ciphertexts are ciphertexts of the underlying MIFE scheme. Finally, the algorithm runs the MIFE evaluation procedure on these ciphertexts using $\mathsf{MCFE.sk}_f$, to recover and thereby output $f(x_1, \ldots, x_t)$ if all labels match, else output $\perp$.

Correctness For all valid t messages $x_1, \ldots, x_t$ under some label lb and for all function f from $\mathcal{F}_\kappa$, correctness of MCFE follows from the correctness of MIFE.

Instantiation Instantiating our construction with the adaptively secure MIFE scheme by Brakerski et al.[BKS18] which when further instantiated with the single-input private-key FE scheme of Gorbunov et al.[GVW12], we obtain a private-key MCFE for constrained domain, supporting bounded keygen and bounded encryption query, by just relying on minimal assumption of one way function (OWF).

5 Proof of Security

The following theorem captures the security of our scheme.

Theorem 1. *For any fixed integer $t > 1$ and security parameter $\kappa \in \mathbb{N}$, consider* MCFE *for $\mathcal{F}$, as given in Figure 2, be defined over input space $\mathcal{X}_\kappa^t$ s.t. $|\mathcal{X}_\kappa| \leq$*

$\mathrm{poly}(\kappa)$ *and label space* $\mathcal{L}_\kappa$ *s.t.* $|\mathcal{L}_\kappa| \leq \mathrm{poly}(\kappa)$. *If* MIFE *is a private-key* (q_1', q_2')*-bounded adaptively secure multi-input functional encryption scheme for* $\mathcal{F}$*, as defined in Figure 1(Left). Then* MCFE *is a private-key* (q_1, q_2)*-bounded adaptively secure multi-client functional encryption scheme for* $\mathcal{F}$*, as defined in Figure 1(Right) such that*

$$q_1 = q_1' \quad \textit{and} \quad q_2 = q_2' - |\mathcal{X}_\kappa| \cdot |\mathcal{L}_\kappa|$$

Proof. Towards proving Theorem 1, let $\mathcal{A}$ be a valid (q_1, q_2)-bounded ppt adversary (as per Definition 5) against adaptively secure MCFE construction from Figure 2. Suppose $\mathcal{A}$ wins the experiment $\mathbf{G}^{\mathrm{adp-ind}}_{\mathsf{MCFE},\mathcal{A}}$ in Figure 1(Right) with non-negligible probability. Using $\mathcal{A}$, we construct a (q_1', q_2')-bounded ppt adversary D, valid as per Definition 2, that breaks the adaptive message security of MIFE in the experiment $\mathbf{G}^{\mathrm{adp-ind}}_{\mathsf{MIFE},\mathrm{D}}$ of Figure 1(Left). Adversary D interacts with the MIFE challenger while internally simulating the MCFE game for $\mathcal{A}$.

In a nutshell, the proof begins with $\mathcal{A}$ submitting the set of corrupted parties, for which D must provide encryption keys comprising of ciphertexts for all plaintext–label pairs associated with those parties. Since D is a MIFE adversary, viewing parties as slots of the t-input function, it first queries its EncO to generate ciphertexts for the corrupted slots, which are then returned to $\mathcal{A}$ as the corresponding encryption keys. Next, $\mathcal{A}$ submits EncO queries for the honest parties. To address these, D embeds the honest queries into their corresponding slots, while filling the corrupted slots with dummy values, assigning identical messages to both challenge positions. The response from the challenger is then forwarded to $\mathcal{A}$.

When $\mathcal{A}$ submits KeygenO queries, D translates directly into its KeygenO queries, with the corresponding responses returned to $\mathcal{A}$. It is easy to see that this is a perfect simulation of $\mathcal{A}$'s view in the MCFE security game. Thus, if $\mathcal{A}$ can distinguish between these ciphertexts, adversary D can break the security of the MIFE scheme. We provide details below.

Initialize: $\mathcal{A}$ begins the game by declaring the corrupt set of parties $\mathcal{CP}$ to D. Now D's job is to generate the corrupt encryption keys. For this, D has to generate ciphertext for all possible plaintext-label pair for the corrupt set of parties by querying its challenge encryption oracle MIFE.EncO. The number of queries that D needs to make to the MIFE.EncO for simulating corrupt keys is $|\mathcal{X}_\kappa| \cdot |\mathcal{L}_\kappa|$.

Technically, to simulate the encryption keys for the corrupted parties, D maintains two lists M^0, M^1 each of size $|\mathcal{X}_\kappa| \cdot |\mathcal{L}_\kappa|$. For every message $m \in \mathcal{X}_\kappa$, and every label $\mathsf{lb} \in \mathcal{L}_\kappa$, $\mathcal{D}$ creates a vector $(m, 1, \mathsf{lb}), (m, 2, \mathsf{lb}), \ldots, (m, t, \mathsf{lb})$ and adds it to both M^0 and M^1. D then queries its encryption oracle MIFE.EncO with M^0, M^1 and receives ciphertexts for each party for each message and label. It then uses the ciphertexts for the parties in $\mathcal{CP}$ as the corrupted ek's and returns these to $\mathcal{A}$.

Encryption Oracle: After obtaining ek_l for each $l \in \mathcal{CP}$, $\mathcal{A}$ issues challenge queries for the honest parties of the form $\{(x_l^0, \mathsf{lb}_l)\}, \{(x_l^1, \mathsf{lb}_l)\}$ for $l \in \mathcal{H}$ and

$\mathcal{A}$ is allowed to make q_2 number of such queries in total. To simulate these, D maintains two sets N^0, N^1 of size q_2 that serve as the challenge inputs for its challenge encryption oracle $\mathsf{MIFE}.\textsc{EncO}$. Each of these queries contain exactly t inputs, one for each slot. Since D has received the inputs only for $\mathcal{H}$ i.e Honest party, he needs to transform each query into $\mathsf{MIFE}.\textsc{EncO}$ query. For each query, D does the following.

- For each honest party $l \in \mathcal{H}$: insert $(x_l^0, l, \mathsf{lb}_l)$ into N^0 and $(x_l^1, l, \mathsf{lb}_l)$ into N^1.
- For each corrupted party $l \in \mathcal{CP}$: choose any $m \in \mathcal{X}_\kappa$ and insert (m, l, lb_l) into both N^0 and N^1.

D then queries its challenge encryption oracle $\mathsf{MIFE}.\textsc{EncO}$ with (N^0, N^1) and receives answers as $\mathsf{MIFE}.\mathsf{ct}_2^\sigma$ where σ is the indistinguishability bit of its challenger. It extracts $\mathsf{MCFE}.\mathsf{ct}_{x_l, \mathsf{lb}_l}^\sigma$ for each $l \in \mathcal{H}$, and forwards these to $\mathcal{A}$.
Recall that q_2' be the total number of encryption oracle queries made to MIFE challenger by D, then from above we can say that

$$q_2' \geq q_2 + |\mathcal{X}_\kappa| \cdot |\mathcal{L}_\kappa|$$

KeyGen Oracle: For each functional key query of form (f^0, f^1) among the q_1 keygen queries issued by $\mathcal{A}$, consider $\mathcal{D}_f$ as the functionality that first checks consistency of labels and then evaluates f on the input plaintexts. For every such query, D invokes its $\mathsf{MIFE}.\textsc{KeygenO}$ on $(\mathcal{D}_{f^0}, \mathcal{D}_{f^1})$ a total of q_1' times. For its challenger's indistinguishability bit σ, it receives answer $\mathsf{MIFE}.\mathsf{sk}_{\mathcal{D}}^\sigma$, and provides it to $\mathcal{A}$ as the simulated MCFE functional key sk_{f^b} . By construction, the admissibility checks in $\mathcal{D}$ ensure consistency with MCFE. The number of key queries is preserved, i.e.,

$$q_1' = q_1.$$

Finalize: Eventually $\mathcal{A}$ outputs a bit b'. D outputs b' as its own guess for the challenge bit σ in the MIFE game. Clearly, if $\mathcal{A}$ wins the MCFE game with non-negligible advantage, then D does so in the MIFE game with the same advantage. Hence, the existence of $\mathcal{A}$ implies the existence of D breaking the adaptive security of MIFE, contradicting its assumed security.

Corollary 1. *For any security parameter κ, assuming one-way functions exist, there exists an adaptively secure private-key* MCFE *scheme for $\mathcal{F}$ over bounded domain with query bounds q_1, q_2 on the number of* KG *and* Enc *queries, respectively.*

Proof. The proof of Corollary 1 follows by instantiating the MIFE in Theorem 1 with the private-key MIFE scheme of Brakerski et al. [BS18], based on the private-key single-input functional encryption scheme of Gorbunov et al. [GVW12].

□

6 Other Related Work

A large body of work has previously studied functional encryption in the multi-input and multi-client settings. We briefly review these papers here organized by the function class they support.

General FE: Research on multi-input general FE was started by Goldwasser et al. [GGG+14], who proposed a construction of MCFE / MIFE based on indistinguishability obfuscation. In terms of expressiveness, their scheme is remarkably powerful, supporting all multi-input functionalities that can be computed by bounded-size circuits. Building on this line of work, Boneh et al. [BLR+15] introduced a private-key MIFE scheme with a stronger notion of security based on multilinear maps. While these papers achieved strong functionality, they both relied on strong assumptions, and moreover proved that these assumptions were necessary in some cases.

A key breakthrough came with the work of Ananth and Jain [AJS15], who started the path towards weakening the assumptions by showing a *selectively-secure* private-key MIFE scheme can be constructed from general-purpose single-input public-key functional encryption. Concurrently, Brakerski et al. [BKS18] gave a private-key MIFE scheme for constant-arity functions under weaker assumptions, including based only on the minimal assumption of one-way functions. However, both of these latter works only constructed MIFE leaving MCFE open for future work.

FE for Restricted Functionalities: Another line of work has instead looked at constructing MIFE/MCFE for restricted function classes. This line was started by Abdalla et al. [AGR+17] who showed how to construct MIFE for inner products from bilinear pairings. Further improvements by Abdalla et al. [ACF+18] developed a more general approach resulting in constructions from DDH, Paillier, and LWE. Later, Agrawal et al. [AGT21] gave a construction of MIFE for quadratic functions.

The first construction of MCFE for inner products was given by Chotard et al. [CDG+18], proven secure under DDH in the random oracle model. Subsequent works obtained standard-model realizations: Abdalla et al. [ABG19] under MDDH, DCR, and LWE assumptions, and Libert et al. [LT19] under LWE. Interestingly, Abdalla et al. [ABK+19] later demonstrated that their earlier MIFE construction [ACF+18] also gives an unlabeled MCFE scheme, thereby providing one of the first explicit connections between the two models. Beyond inner products, Ciampi et al. [CSW21] constructed a MCFE scheme for separable functions.

Further work considered how to achieve function-privacy for MCFE. Agrawal et al. [AGT21] presented the first function-hiding MCFE construction using bilinear pairings, proving its security in the random oracle model. Later, Shi and Vanjani [SV23] refined this approach to eliminate reliance on the random oracle model. Notably, all known function-hiding labeled MCFE schemes rely on pairings.

Acknowledgments

Arkady Yerukhimovich is supported in part by NSF grants CNS-2144798 (CAREER) and CNS-1955620.

References

[ABG19] Michel Abdalla, Fabrice Benhamouda, and Romain Gay. "From single-input to multi-client inner-product functional encryption". In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2019, pp. 552–582.

[ABK+19] Michel Abdalla, Fabrice Benhamouda, Markulf Kohlweiss, and Hendrik Waldner. "Decentralizing inner-product functional encryption". In: *IACR International Workshop on Public Key Cryptography*. Springer. 2019, pp. 128–157.

[ACF+18] Michel Abdalla, Dario Catalano, Dario Fiore, Romain Gay, and Bogdan Ursu. "Multi-input functional encryption for inner products: Function-hiding realizations and constructions without pairings". In: *Annual International Cryptology Conference*. Springer. 2018, pp. 597–627.

[AGR+17] Michel Abdalla, Romain Gay, Mariana Raykova, and Hoeteck Wee. "Multi-input inner-product functional encryption from pairings". In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2017, pp. 601–626.

[AGV+13] Shweta Agrawal, Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. "Functional encryption: New perspectives and lower bounds". In: *Annual Cryptology Conference*. Springer. 2013, pp. 500–518.

[AGT21] Shweta Agrawal, Rishab Goyal, and Junichi Tomida. "Multi-input quadratic functional encryption from pairings". In: *Annual International Cryptology Conference*. Springer. 2021, pp. 208–238.

[AJS15] Prabhanjan Ananth, Abhishek Jain, and Amit Sahai. "Indistinguishability obfuscation from functional encryption for simple functions". In: *Cryptology ePrint Archive* (2015).

[BGI+01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil Vadhan, and Ke Yang. "On the (im) possibility of obfuscating programs". In: *Annual International Cryptology Conference*. Springer. 2001, pp. 1–18.

[BO13] Mihir Bellare and Adam O'Neill. "Semantically-secure functional encryption: Possibility results, impossibility results and the quest for a general definition". In: *International Conference on Cryptology and Network Security*. Springer. 2013, pp. 218–234.

[BV18] Nir Bitansky and Vinod Vaikuntanathan. "Indistinguishability obfuscation from functional encryption". In: *Journal of the ACM* 65.6 (2018), pp. 1–37.

[BLR+15] Dan Boneh, Kevin Lewi, Mariana Raykova, Amit Sahai, Mark Zhandry, and Joe Zimmerman. "Semantically secure order-revealing encryption: Multi-input functional encryption without obfuscation". In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2015, pp. 563–594.

[BSW11] Dan Boneh, Amit Sahai, and Brent Waters. "Functional encryption: Definitions and challenges". In: *Theory of Cryptography Conference*. Springer. 2011, pp. 253–273.

[BW07] Dan Boneh and Brent Waters. "Conjunctive, subset, and range queries on encrypted data". In: *Theory of Cryptography Conference*. Springer. 2007, pp. 535–554.

[BCP14] Elette Boyle, Kai-Min Chung, and Rafael Pass. "On extractability obfuscation". In: *Theory of Cryptography Conference*. Springer. 2014, pp. 52–73.

[BKS18] Zvika Brakerski, Ilan Komargodski, and Gil Segev. "Multi-input functional encryption in the private-key setting: Stronger security from weaker assumptions". In: *Journal of Cryptology* 31.2 (2018), pp. 434–520.

[BS18] Zvika Brakerski and Gil Segev. "Function-private functional encryption in the private-key setting". In: *Journal of Cryptology* 31.1 (2018), pp. 202–225.

[CDG+18] Jérémy Chotard, Edouard Dufour Sans, Romain Gay, Duong Hieu Phan, and David Pointcheval. "Decentralized multi-client functional encryption for inner product". In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2018, pp. 703–732.

[CSW21] Michele Ciampi, Luisa Siniscalchi, and Hendrik Waldner. "Multi-client functional encryption for separable functions". In: *IACR International Conference on Public-Key Cryptography*. Springer. 2021, pp. 724–753.

[DIJ+13] Angelo De Caro, Vincenzo Iovino, Abhishek Jain, Adam O'Neill, Omer Paneth, and Giuseppe Persiano. "On the achievability of simulation-based security for functional encryption". In: *Annual Cryptology Conference*. Springer. 2013, pp. 519–535.

[GGH+16] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. "Candidate indistinguishability obfuscation and functional encryption for all circuits". In: *SIAM Journal on Computing* 45.3 (2016), pp. 882–929.

[GGG+14] Shafi Goldwasser, S Dov Gordon, Vipul Goyal, Abhishek Jain, Jonathan Katz, Feng-Hao Liu, Amit Sahai, Elaine Shi, and Hong-Sheng Zhou. "Multi-input functional encryption". In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2014, pp. 578–602.

[GKP+13] Shafi Goldwasser, Yael Kalai, Raluca Ada Popa, Vinod Vaikuntanathan, and Nickolai Zeldovich. "Reusable garbled circuits and

succinct functional encryption". In: *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*. 2013, pp. 555–564.

[GVW12] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. "Functional encryption with bounded collusions via multi-party computation". In: *Annual Cryptology Conference*. Springer. 2012, pp. 162–179.

[GKL+13] S Dov Gordon, Jonathan Katz, Feng-Hao Liu, Elaine Shi, and Hong-Sheng Zhou. "Multi-input functional encryption". In: *Cryptology ePrint Archive* (2013).

[GKL+15] S Dov Gordon, Jonathan Katz, Feng-Hao Liu, Elaine Shi, and Hong-Sheng Zhou. "Multi-client verifiable computation with stronger security guarantees". In: *Theory of Cryptography Conference*. Springer. 2015, pp. 144–168.

[GPS+06] Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. "Attribute-based encryption for fine-grained access control of encrypted data". In: *ACM CCS*. 2006, pp. 89–98.

[HEK12] Yan Huang, David Evans, and Jonathan Katz. "Private set intersection: Are garbled circuits better than custom protocols?" In: *NDSS*. 2012.

[KSW08] Jonathan Katz, Amit Sahai, and Brent Waters. "Predicate encryption supporting disjunctions, polynomial equations, and inner products". In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2008, pp. 146–162.

[KNT18] Fuyuki Kitagawa, Ryo Nishimaki, and Keisuke Tanaka. "Obfustopia built on secret-key functional encryption". In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2018, pp. 603–648.

[KMN+14] Ilan Komargodski, Tal Moran, Moni Naor, Rafael Pass, Alon Rosen, and Eylon Yogev. "One-way functions and (im) perfect obfuscation". In: *IEEE FOCS*. 2014, pp. 374–383.

[LOS+10] Allison Lewko, Tatsuaki Okamoto, Amit Sahai, Katsuyuki Takashima, and Brent Waters. "Fully secure functional encryption: Attribute-based encryption and (hierarchical) inner product encryption". In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2010, pp. 62–91.

[LT19] Benoît Libert and Radu Titiu. "Multi-client functional encryption for linear functions in the standard model from LWE". In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2019, pp. 520–551.

[NPP22] Ky Nguyen, Duong Hieu Phan, and David Pointcheval. "Multi-client functional encryption with fine-grained access control". In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2022, pp. 95–125.

[NPP23] Ky Nguyen, Duong Hieu Phan, and David Pointcheval. "Optimal security notion for decentralized multi-client functional encryption". In: *International Conference on Applied Cryptography and Network Security*. Springer. 2023, pp. 336–365.

[NPP25] Ky Nguyen, Duong Hieu Phan, and David Pointcheval. "Multi-client functional encryption with public inputs and strong security". In: *IACR International Conference on Public-Key Cryptography*. Springer. 2025, pp. 68–101.

[NPS24] Ky Nguyen, David Pointcheval, and Robert Schädlich. "Decentralized multi-client functional encryption with strong security". In: *Cryptology ePrint Archive* (2024).

[ONe10] Adam O'Neill. "Definitional issues in functional encryption". In: *Cryptology ePrint Archive* (2010).

[SW05] Amit Sahai and Brent Waters. "Fuzzy identity-based encryption". In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2005, pp. 457–473.

[SV23] Elaine Shi and Nikhil Vanjani. "Multi-client inner product encryption: Function-hiding instantiations without random oracles". In: *IACR International Conference on Public-Key Cryptography*. Springer. 2023, pp. 622–651.